

Capítulo 1: Introducción a Sistemas Distribuidos

Capítulo 2: Arquitectura en Sistemas distribuidos

Capítulo 3: Comunicación en Programación distribuida

Capítulo 4: Fundamentos teóricos de la computación distribuida

0.1 Historias Causales

Las **Historias causales** permiten detectar si existe **Concurrencia** o **Causalidad** entre dos eventos.

■ **C :** **Contenido** $\Rightarrow$: $A = \{1, 2\}$ y $B = \{1, 2, 3\}$ podemos decir que $A \subset B$

■ **A** de lo antes definido tenemos lo siguiente:

$\forall e, e' \in E, e \neq e' : [e' \rightarrow e] \Rightarrow H(e') \subset H(e)$

■ Esto se puede traducir como: Si e' ocurre antes que e , entonces $H(e')$ es un subconjunto de $H(e)$.

■ $\forall e, e' \in E, e \neq e' : [e' \parallel e] \Leftrightarrow [H(e') \not\subseteq H(e)] \wedge [H(e) \not\subseteq H(e')]$

■ Esto se lee como: Si e' es concurrente con e , entonces $H(e')$ no es un subconjunto de $H(e)$ y $H(e)$ no es subconjunto de $H(e')$.

0.2 Relojes vectoriales

Definiremos la siguiente estructura para un proceso P_i , para n procesos.

$[Vc[1], Vc[2], \dots, Vc[i], \dots, Vc[n]]$

Cada índice este asociado a un proceso P_i . Sabemos que $\forall e \in E: Vc(e)$ representa $H(e)$. Los procesos irán aumentando de la siguiente forma:

■ Para **eventos locales** del proceso P_i :

Paso 1: $Vc(e_i[j] + 1)$, es decir que aumentamos la marca del proceso i

Paso 2: Asignar el valor anterior al resto de los otros procesos, al nuevo evento.

$[Vc[1], Vc[2], \dots, Vc[i], \dots, Vc[n]]$

donde los índices en verde, se mantendrán iguales para el evento local.

Resultando en el siguiente vector para P_i

$[Vc[1], Vc[2], \dots, Vc[i] + 1, \dots, Vc[n]]$

■ Para mensajes entre procesos solo se modifica la marca del proceso que está mandando el mensaje. Por ejemplo, si P_i manda un mensaje a P_j , se modifica la marca del vector P_j asociada a P_i .

0.2.1 Propiedades de los relojes vectoriales

Lo siguiente que denotaremos es **condición fuerte de un reloj en sistemas distribuidos** se refiere a la capacidad de determinar de forma bidireccional si un evento precede causalmente a otro, permitiendo un ordenamiento total o casi total de los eventos en el sistema.

1. Relación de igualdad:

$V = V' \equiv [\forall k : V[k] = V'[k]]$

2. Relación de orden:

$V < V' \equiv [V \neq V'] \wedge [\forall k : V[k] \leq V'[k]]$

3. Condición fuerte:

$e \rightarrow e' \equiv VC(e) < VC(e')$

$e \parallel e' \equiv \neg[VC(e) < VC(e')]$

$\wedge \neg[VC(e') < VC(e)]$

El numero de eventos precedentes es equivalente a la suma del vector completo, menos el evento mismo.

La propiedad de detección de apertura nos permite a un proceso P_j detectar que existe un evento e' que ocurre en otro P_i después de un evento e_i ya conocido por P_j . Notese además que los relojes vectoriales tienen la capacidad de **detectar causalidad, concurrencia, coordinación centralizada y es tolerante a fallos**. Sin embargo tiene limitaciones, como, **overhead, complejidad (más complejo que lamport)**, si bien dijimos que tolera fallos, es deficiente ante fallos bizantinas y sistemas altamente dinámicos (con una entrada y salida de procesos, dado que se tiene que redimensionar el vector)

0.3 Monitoreo de una computación distribuida

Una **observación válida** si implica una relación causal, debería estar ordenada, caso contrario, no (esta se podría ver de cualquier forma)



Figura 1: Monitoreo de computación distribuida

La cola observada en la figura anterior, es la encargada de generar las observaciones válidas. Para entregas causales tenemos las siguientes condiciones:

■ **Condición FIFO:**

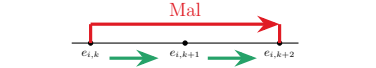


Figura 2: Ejemplo de relaciones entre eventos

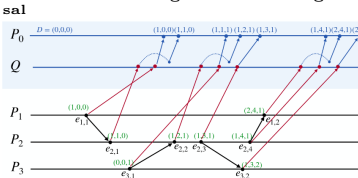
Las líneas verdes hacen el flujo de izquierda a derecha que representa una correcta condición FIFO, mientras que las rojas omiten el evento $e_{i,k+1}$ lo cual muestra una incorrecta condición fifo.

■ **Condición de entrega causal:** Si yo tengo un mensaje de J, para ver que no exista un men-

saje de otro proceso, se compara lo siguiente

$\forall k, i \neq j : Vc_i(m') [k] \geq Vc_j(m) [k]$

Visualización del algoritmo de entrega causal



garantizar la consistencia de los datos o mantener el correcto funcionamiento de un sistema distribuido.

Una solución correcta para el problema de la exclusión mutua considera los siguientes estados y requisitos

- a) Estados de un proceso:
- P_i respecto a la SC: **Requesting**: P_i está solicitando ingresar a la SC; y se bloquea si SC está ocupada por otro proceso. **Executing**: P_i está ejecutando la SC y ningún otro proceso puede estar simultáneamente en este estado. **Idle**: P_i está fuera de la SC, ejecutando acciones independientes; se dice que el proceso está ocioso.
- b) Requisitos para la solución:
- Seguridad**: En todo momento a lo más un proceso está en la SC. **Vivacidad**: La solicitud a entrar en la SC se resuelve en tiempo finito. **Justicia**: Si una solicitud de ingreso a la SC sucede antes que otra, entonces la entrada a la SC es en ese orden. Eventualmente podrán agregarse requisitos de rendimiento y resistencia a fallas.

0.8.1 Exclusión mutua con control centralizado

Caracterización: Requiere de un único proceso controlador que otorga el permiso para entrar a la SC. Procesos que solicitan permiso al controlador deben esperar respuesta para ingresar a su SC. La solución requiere 3 tipos de mensajes:

- Request**: Proceso solicita permiso a controlador para ingresar a su SC.
- Grant**: Controlador concede el permiso.
- Release**: Proceso que sale de su SC, avisa al controlador sobre su liberación.

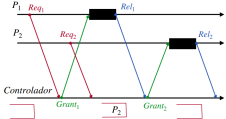


Figura 8: Esquema de exclusión mutua con control centralizado

0.9 Algoritmos basados en token

0.9.1 Algoritmo de Lamport

Caracterización: Algoritmo descentralizado basado en permisos, para un sistema asincrónico y basado en relojes lógico de lamport, para ordenar las solicitudes de ingreso a la SC.

Condiciones para cada proceso P_i :

Mantiene un reloj lógico LC y marca sus solicitudes con el rel (TS_i, i), para establecer un orden global único.

Comunicación es por paso de mensajes asincrónico y garantiza orden FIFO para entrega de los mensajes.

Mantiene cola **RequestQueue_i** con solicitudes ordenadas ascendentemente por marca de tiempo. $R_i = \{P_1, P_2, \dots, P_n\} : \forall i, 1 \leq i \leq n$

Tipos de mensajes: Request, Reply, Release.

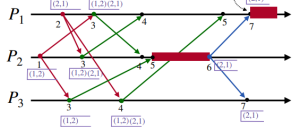


Figura 9: Esquema del algoritmo de Lamport

Desempeño: la **3(N-1)** mensajes por cada invocación a la SC. **Retardo de sincronización**: T . **Throughput** = $\frac{1}{3+T}$

0.9.2 Algoritmo de Ricart y Agrawala

Claves de diseño: Comparte la misma caracterización general con el algoritmo de Lamport. Cada proceso requiere permiso de todos los demás procesos. Inicialmente se cumple $\forall i, 1 \leq i \leq n$: $State_i = IDLE \wedge RequestQueue_i = \emptyset$. La RequestQueue contiene a los procesos que han sido diferidos en el ingreso a la SC. Solo se requiere 2 tipos de mensajes: Request y Reply Con Ricart-Agrawala, el Reply (Verde en la figura) hace el trabajo de Grant (para notificar el próximo proceso que tenga acceso a la SC), eso lo hace retrasando en envió de **reply** hasta el final.

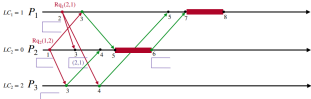


Figura 10: Esquema del algoritmo de Ricart y Agrawala

Desempeño: 2(N-1) mensajes por cada invocación a la SC. **Retardo de sincronización**: T . **Throughput** = $\frac{1}{2+T}$

0.9.3 Algoritmo de Suzuki-Kasami

Caracterización: Algoritmo descentralizado basado en token, para un sistema asincrónico. Garantiza seguridad, vivacidad y justicia.

Características específica: Un único token circula entre los procesos que lo solicitan y únicamente el proceso en posición del token puede hacer uso de SC, **garantizando exclusión mutua**. Reduce la sobrecarga de mensajes; sin embargo, la pérdida del token interrumpe operación del sistema, lo que **requiere mecanismos de recuperación**.

0.9.4 Idea general de funcionamiento

Supondremos N procesos. Un proceso $P_i = (\forall i, 1 \leq i \leq N)$ que intenta regresar a la SC y que no tiene token, debe solicitarlo a los demás procesos. Difunde un mensaje REQUEST por el token a todos los demás procesos. El proceso que posee el token al recibir mensaje REQUEST, se lo envía al solicitante bajo las siguientes condiciones:

- El token se envía al solicitante sólo si el proceso en posesión de éste no está en la SC
 - Si el proceso con el token estuviera en su SC, para ella retardar su envío hasta que salga de la SC.
 - En el caso de existir varias solicitudes pendientes, se prioriza acceso que primero generó la solicitud.
- El token es una estructura de datos que lleva vector de registro de las solicitudes procesadas para cada proceso y mantiene una cola de solicitudes pendientes, estableciendo un orden de procesamiento y garantizando justicia y vivacidad.

0.9.5 Claves de diseño

- a) **Identificador único de solicitudes**: Cada solicitud de un proceso P_i se identifica con su UID y un número de secuencia ns_i independiente de otros procesos. Luego, cada mensaje de solicitud queda únicamente identificada como: $REQUEST(i, ns_i)$
- b) **Arreglo de solicitudes por proceso**: P_i mantiene $RN_i[1 \dots N]$, donde $RN_i[j]$ es la mayor solicitud recibida por P_i desde P_j . Permite distinguir en el sistema entre solicitudes ajenas y solicitudes recientes de un mismo proceso.
- c) **Token**: Es único en el sistema y consiste de una **cola Q** y un arreglo de solicitudes completamente procesadas $LN[1 \dots N]$. **Q** corresponde a una cola FIFO que **ordena** el ingreso a la SC de procesos que solicitan necesitan el token. $LN[1 \dots N]$ es un arreglo, donde $LN[i]$ contiene ns_i de última solicitud procesada para proceso P_i permite determinar qué procesos tienen solicitudes pendientes.
- d) **Lógica de procesos**: Un proceso P_i en posición del token al salir de su SC, lo actualiza antes de considerar su envío haciendo: P_i actualiza en el token: $LN[i] = RN[i]$. Si $RN_i[j] = LN[j] + 1$ y P_j no está en la cola, entonces **P_j tiene una solicitud pendiente y debe ser agregado a Q**

Desempeño: Genera 0 a N mensajes por solicitud procesada, tiempo de sincronización: 0 o T. **Throughput** máximo = $\frac{1}{2+T}$

El orden de agregar las solicitudes pendientes a la cola FIFO del token, es normalmente por nuevas solicitudes **ordenadas por el identificador de proceso**. El orden de procesamiento **no es necesariamente por orden de llegada**. La FIFO garantiza que no exista postergación indefinida.

Capítulo 6: Confiabilidad y tolerancia a fallos

Confiabilidad: La confianza que se puede tener en un sistema, aun en presencia de errores, fallos y condiciones inesperadas.

Atributos claves

- Fiabilidad**: Continuidad correcta del servicio sin fallos o interrupciones.
- Disponibilidad**: La disposición de un servicio a ser usado en cualquier tiempo T.
- Mantenibilidad**: Facilidad de mantener, actualizar y reparar un sistema que puede fallar.
- Protección**: Ausencia de consecuencias catastróficas sobre el medio ambiente.
- Seguridad**: Prevenir el acceso o manipulación no autorizada (alineado con la **CIA: Confidencialidad, Integridad y Disponibilidad**).

Los atributos antes mencionados son cuantificables y medibles para el análisis de la **tolerancia a fallos**.

0.10 Modelo Causal de averías

Para este contexto definiremos al sistema como un conjunto de componentes tanto de hardware como de software, preparados para colaborar coordinadamente para ofrecer un servicio, según pautas establecidas para un correcto funcionamiento.

Pero, que causa un mal funcionamiento de un sistema. Partiremos definiendo 3 diferencias claves para el análisis.

- Fallo(fault)**: Defecto o funcionamiento anormal de un componente, el cual tiene el potencial de causar un error, en caso de no ser tratado **puede llevar a una avería**. Se clasifican según persistencia (**transiente, intermitente o permanente**) y según causa (**error de manufactura, diseño u operacional**)
- Error**: Discrepancia entre el comportamiento esperado y el exhibido
- Avería(failure)**: Ocurre un malfuncionamiento que conduce a que ya no se pueda proveer el servicio especificado. Son causas para errores que se propagan a lo largo del sistema, llegando a ser observables por el usuario.

Un error o fallo no necesariamente conduce a una avería, si este es tratado y manejado adecuadamente

Las **fallas o averías** se clasifican según su comportamiento en el sistema

- Fallas de **caída (crash)**: Componente se detiene y no se recupera por sí mismo
- Proceso**: Proceso fallado se detiene, puede perder su estado. Es la mas benigna es detenerse manteniendo su estado, para posteriormente recuperarlo. Para la detención fiable por parte de otros procesos se requieren **timers**
- Comunicación**: Falla del sistema de comunicación con pérdida continua de mensajes (e.g falla permanente o partición de red)
- Fallas de **omisión**: El sistema no responde transitoriamente
- Proceso**: Proceso no responde, sin perder estado (e.g., se pierde una solicitud de servicio).
- Comunicación**: Se pierden algunos mensajes. Falla mas benigna es que mensajes no se corrompen (integridad) y no se puede falsear su remitente (autenticidad). Recuperación es por retransmisión de mensajes perdidos.
- Fallas **temporales (timing)**: El sistema no responde dentro de los límites de tiempo. Pueden corresponder a un mal desempeño por sobrecarga.
- Pueden ocurrir por fallas de relojes.
- Fallas **arbitrarias (bizantinas)**: El sistema se

comporta inconsistente o maliciosamente.

Los procesos fallas siguen funcionando pero no garantizan la integridad de los mensajes (**peor caso**)

A veces se produce por corrupción interna del sistema, causan respuestas erróneas pero no maliciosas

Son fallas difíciles de detectar y tolerar

Una vez definidas el tipo de fallas, establezcamos los modelos para los tipos de fallas, en base a una jerarquía (esta ordenada de adentro hacia afuera, vale decir, de la más "debil,a la más dañina):

- Fail-stop**: Un proceso fallado se detiene, conservando su estado, y todos los demás saben inmediatamente que ha fallado. Los procesos se comportan correctamente o se detienen.
- Crash**: Modelo realista de incertidumbre para sistemas asincrónicos. Un proceso fallado se detiene, conserva su estado, pero los demás no lo saben ni están seguros si se cayó o esta lento o tiene problemas de comunicación. Puede recuperarse y reintegrarse al sistema.
- Omisión**: Solo un proceso puede omitir enviar o recibir mensajes. Se puede abordar con el uso de timers, reintento y asentimientos.
- Temporal (timing)**: El sistema puede violar restricciones temporales, tal como respuestas fuera de tiempo. Difícil de discriminar entre fallas de crash y omisión
- Bizantino**: Un proceso puede enviar mensajes contradictorios, mentir sobre su estado interno o colaborar maliciosamente con otros. Corresponde a un modelo más general



Figura 11: Modelo Causal de averías

Para manejar lo anterior, se propone los siguientes enfoques para mejorar la confiabilidad del sistema.

- Prevención de fallos**: Prevenir que ocurran se interrumpan los fallos del sistema
- Remoción de fallos**: Durante el desarrollo o durante el uso (**mantención y reparación**)
- Pronóstico de fallos**: Predecir fallos probables, para que puedan ser anticipadamente eliminados o contener sus efectos (e.g. **alarmas y detección de degradación/anomalías**)
- Pronóstico de fallos**: Capacidad de un sistema para seguir funcionando normalmente, aun cuando se produzcan uno o más fallos de sus componentes

Se debe disponer de algún tipo de redundancia para enmascarar fallos.

Capacidad de reparar un fallo sin interrumpir el servicio, para asegurar continuidad operacional del servicio.

0.11 Tolerancia a fallos

La tolerancia a fallos es la capacidad de enmascarar la existencia de fallos en un sistema, usando redundancia. Esto busca evitar una avería o falla de sistema. Si bien un sistema puede ser tolerante a fallos, estos deberían ser de sus propios componentes, y no con fallas que comprometen todo el sistema. Queremos que el sistema, tenga un comportamiento consistente a lo especificado.

El manejo de fallos en un sistema es un proceso de cuatro fases: primero, la **Detección de error**, donde los fallos se deducen por errores en el estado del sistema; segundo, la **Aislación y evaluación del daño**, que busca contener la expansión del error; tercero, la **Recuperación de error**, que corrige el problema y devuelve el sistema a un estado consistente (e.g., **rollback**); y finalmente, el **Tratamiento del fallo y servicio continuo**, que, ante fallos permanentes, reconfigura el sistema para dejar de usar el componente dañado y reemplazarlo.

La **redundancia** se define en base a las partes de un sistema que no son necesarias para su correcto funcionamiento, si no se requiere tolerar fallos. Existen 3 tipos de tolerancias

- Espacial**: Se agregan componentes de redundantes de hardware o software.
- Temporal**: Ejecutar y repetir varias veces una acción o secuencia de instrucciones. Un ejemplo es la retransmisión de mensajes, rollback de una acción y repetir su ejecución.
- De información**: Se agrega información redundante para tolerar fallos.

0.12 Enfoques de recuperación de errores

Recuperación regresiva (backward recovery): El sistema retrocede a un estado correcto anterior y se repite la ejecución, se introduce el concepto de checkpoints en la ejecución, los cuales son puntos donde se guarda el estado del sistema en una memoria estable (no se ve afectada por alguna avería). En caso de detectar errores, el sistema hace un rollback.

Recuperación progresiva (forward recovery): El sistema sigue avanzando, corrigiendo un posible estado erróneo. Al detectar errores, se sigue la ejecución mientras que a la par se realizan acciones correctivas. No es viable para todos los sistemas

Criterio	Recuperación regresiva	Recuperación progresiva
	- Más simple de implementar - Independencia del tipo de fallo o error (no requiere diagnóstico tipo de fallo que causa el error)	Permite seguir funcionando, sin perder tiempo y diagnóstico al interrumpir el servicio.
Desventajas	- Requiere de trabajo de tiempo al volver atrás - Algunos tipos de errores (checkpoints) pueden ser ineficientes si el estado es grande. - No garantiza la recuperación completa. - Algunos componentes o acciones no son recuperables.	- Más complejo de diseñar y realizar: requiere de verificación de errores. - Requiere prever los posibles fallos o errores. - Aplicable sólo en algunos casos, por lo tanto, no es generalizable.
Ejemplos y mecanismos	- Rollback a log de mensajes - Independencia de grandes volúmenes de datos. - Procesamiento de larga duración.	- Replicación activa, votaciones o acuerdos - Replicación de datos y discos RAID - Existencia de tiempo y de modo crítico - Sistemas con auto-sano

Figura 12: Comparativa de enfoques de recuperación

Esquemas para tolerancia a fallos

- Transacciones o acciones atómicas
- Checkpoint y rollback
- Replicación de procesos
- Máquina de estado replicada o determinista: Basada en multicast atómico con orden total. Ordenamiento basado en consenso (Paxos, Raft)

0.13 Máquina de estado determinista y replicada

Grupo de replicación, donde cada proceso no fallado ejecuta los mismos comandos en el mismo orden. Se caracteriza por el determinismo de estado. donde todas las replicas correctas mantienen el mismo estado Se necesita consenso (dinámico) de los miembros del grupo, este consenso es sobre **cúales comandos ejecutar y en que orden**. Si se trabaja en un modelo **sin fallas, se pueden usar relojes de lamport para ordenamiento global y total de los mensajes del grupo** con un secuenciador central. Para el caso del modelo con fallas, se requieren de algoritmos de consenso tolerantes a fallos.

Vamos a considerar lo siguiente, un contador k, el cuál es iniciado con un valor de 1, tenemos dos máquinas que ejecutarán las mismas instrucciones, pero en diferente orden:

Máquina A: $k+1$ | **Máquina B:** $k+2$

Esto resulta en que el contador de la máquina A, es $k = 3$ y para la máquina B es $k = 4$. Buscamos mantener un orden y los mismos comandos para ambas máquinas.

0.14 Modelos cuantitativos de confiabilidad

Página 21 Se define la **fiabilidad R(t)** como la probabilidad de que un sistema no falle hasta el tiempo t. Definimos **MTTF (Mean Time To Failure)** como el **tiempo de vida esperado sin un fallo del sistema**:

$$MTTF = \int_0^{\infty} R(t) dt$$

Caso especial: donde $R(t)$ sigue una **distribución exponencial** tenemos que $R(t) = \exp -\lambda t$ y $MTTF = \frac{1}{\lambda}$

0.14.1 Fiabilidad en serie

SepiaEs aquel en donde si un componente falla el sistema completo falla. Donde $R_i(t)$ es la fiabilidad del componente i, y λ_i es la tasa media de fallas. La fiabilidad del sistema es:

$$R_{serie} = \prod_{i=1}^n R_i(t)$$

Caso especial, donde $R(t)$ sigue una distribución exponencial tenemos que $R(t) = \exp -\lambda t$ y $MTTF_{serie} = \frac{1}{\lambda}$ con $\lambda = \sum_{i=1}^n \lambda_i$

0.14.2 Fiabilidad en paralelo

Es aquel donde si fallan **TODOS** los componentes, el sistema falla.

$$R_{paralelo} = 1 - \prod_{i=1}^n (1 - R_i(t))$$

Caso especial, donde $R(t)$ sigue una distribución exponencial tenemos que $R(t) = \exp -\lambda t$ y $MTTF_{paralelo} = \frac{1}{\lambda} * \frac{\ln(n)}{\lambda}$ con $\lambda = \sum_{i=1}^n \lambda_i$

0.14.3 Caso de fiabilidad en discos raid-4 y espejo



Figura 13: Ejemplo disco espejo

Para este caso el $MTTF \approx \frac{1}{2} * \frac{MTTF^2}{MTTR}$

Figura 14: Ejemplo para discos fragmentados RAID-4

Para este caso el $MTTF \approx \frac{1}{n(n-1)} * \frac{MTTF^2}{MTTR}$. El funcionamiento de este ultimo considera que se almacenan grupos de K bloques de datos y se el uso de modulo dos (XOR) para el calculo de paridad. De lo anterior, se define lo siguiente para el calculo del bloque de datos D_i , y P como la paridad del sistema.

$$D_i = D_1 \oplus D_2 \oplus \dots \oplus D_{i-1} \oplus D_{i+1} \oplus \dots \oplus D_k \oplus P$$

0.15 Disponibilidad

Disponibilidad instantánea como $A(T)$ como la probabilidad de que un sistema esté funcionando correctamente en un tiempo T. Disponibilidad límite, se define como un valor a , como la disponibilidad promedio en un intervalo $[0, \tau]$:

$$a = \lim_{\tau \rightarrow \infty} \frac{1}{\tau} \int_0^{\tau} A(t) dt$$

Aca se comienza a hablar sobre el concepto de reparacion, de la cual definimos el siguiente "flujo"(Notar que ahora un componente tiene 2 estados, reparándose y funcionando). de la cual se extrae lo siguiente:



Figura 15: Flujo de vida de un componente

El MTTF corresponde al "Mean Time To Failure", MTBF como "Mean Time Between Failure". MTTR como "Mean Time To Repair", tenemos que $MTBF = MTTF + MTTR$. Entonces la disponibilidad límite a como:

$$a = \frac{MTTF}{MTTF + MTTR}$$

0.15.1 Caso de replicación de servidores

Suponiendo un sistema de n réplicas similares, donde la disponibilidad por servidor es de a_0 , trabajando con un modelo en paralelo, la solución de disponibilidad es $a_n = 1 - (1 - a_0)^n$

0.16 Acuerdo y consenso distribuido

El consenso busca garantizar la consistencia y confiabilidad frente a fallas (de nodos, red o bizantinas) y asincrónica. El **objetivo es alcanzar una decisión común** satisfaciendo propiedades de **safety** (Acuerdo, Integridad, Validez) y **liveness** (Término).

En el problema del acuerdo, se busca decidir sobre un único valor o curso de acción tal que se satisficiera el acuerdo (el valor decidido debe ser igual para todos), **integridad** (no se puede cambiar la decisión de un participante), **validez** (el valor elegido debió ser propuesto por alguno de los participantes) y **término** (se decide en tiempo finito). Notar que las tres primeras condiciones hacen referencia a **seguridad** y la última a **terminación**.

- Acuerdo:** Existe un proceso designado (transmisor) que tiene y propone un valor inicial, para alcanzar acuerdo con los demás procesos sobre este valor, según las siguientes condiciones:
- **Acuerdo:** Un proceso transmisor propone un valor. Todos los procesos correctos deben acordar el mismo valor (si el transmisor es correcto, se acuerda su valor inicial).
 - **Consenso:** Cada proceso tiene un valor inicial. Todos los procesos correctos acuerdan un único valor común (si todos tenían el mismo valor inicial, ese debe ser el elegido).
 - **Consistencia Interactiva:** Todos tienen un valor inicial. Se acuerda un vector de valores A_i , donde $A[i]$ corresponde al valor inicial del proceso P_i (si este no ha fallado).

Para garantizar solución, se requiere una relación de fallas $f < n$ y comunicación fiable. En sistemas puramente asíncronos es imposible alcanzar una solución determinista, dado que no se puede distinguir entre una demora y una falla.

0.16.1 Algoritmo de acuerdo bizantino con mensajes ordinarios (no tienen ningún hash criptográfico)

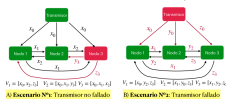


Figura 16: Ejemplo de OM con $n=4$ y $f=1$

En este algoritmo, sólo se trabaja con un modelo de fallas bizantinas y no de comunicación, donde se debe cumplir que $3f + 1 \leq n$. Para el número m de rondas, se cumple que existe a lo más $m = f + 1$ rondas.

0.17 Comunicación grupal tolerante a fallos

El principal enfoque es: una vez tengamos el mecanismo Fuchsianecesitamos las herramientas (replicación de máquina de estado, primary-backup, sistemas basados en quorums) para la replicación de los datos.

- Dos tipos de multicast:
- Simple: No está determinado el número de receptores efectivos, cuando existen emisores de uno o varios no está determinado el orden de llegada a los receptores, es útil para aplicaciones no críticas. No tiene condiciones de consistencia, es más eficiente o liviano.
 - Fiable: Garantiza la entrega a procesos no fallados estando en presencia de fallos de comunicación. Se retransmiten mensajes automáticamente, eliminando mensajes repetidos o corruptos.

0.17.1 Semántica de entrega de mensajes

Fiable: Entrega de mensajes

- **Mejor esfuerzo (Best-effort):** Sin garantía sobre cuántos miembros reciben un mensaje.
 - **Fideliad-k:** Garantía de que a menos k miembros reciben el mensaje.
 - **Atómico:** Garantía de que todos los miembros lo reciben, o ninguno de ellos lo recibe.
- Orden de entrega:
- **No ordenado:** Ninguna garantía de orden de entrega en el receptor.
 - **Orden FIFO:** Mensajes de un mismo emisor se entregan según orden de envío.

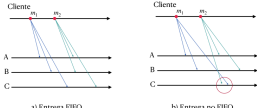


Figura 17: Multicast: orden FIFO

- **Orden causal:** Se respeta en la entrega el orden causal de eventos de envío.

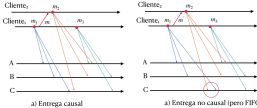


Figura 18: Multicast: orden causal

- **Orden total:** Único orden de entrega de los mensajes en los receptores. Podría requerir también cumplir orden causal.

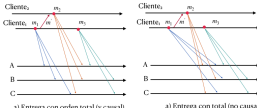


Figura 19: Multicast: orden total

Obs.1: Es necesario utilizar multicast con entrega con orden total y causal para máquinas de estado replicadas deterministas para asegurar la entrega de comandos en el mismo orden. En los grupos muchas veces existen miembros distinguidos (con un rol asociado), los nodos deben tomar el rol del nodo afectado una vez detectada la falla. Si en algún punto un nuevo nodo se integra al grupo, se debe sincronizar con los demás nodos.

0.17.2 Modelo de sincronía virtual (modelo Fail-stop)

La sincronía virtual es un modelo de comunicación que asegura consistencia de entrega de mensajes estando también en presencia de fallos. Es fácil de implementar una máquina de estado replicada y determinista y replicada en un modelo Fail-stop multicast fiable y cambios de membresía. El modelo no soporta fallas bizantinas ni particiones de red. El sistema es parcialmente asíncrono dado que requiere de un timer para detectar la falla

de procesos.

ABCAST: Asegura orden total, tiene la semántica de entrega más fuerte. Para que un grupo reciba un mensaje en el mismo orden se necesita un protocolo de dos fases. El protocolo que manda un mensaje actúa como coordinador, va a actuar en el grupo para determinar el orden en que deben de llegar los mensajes desde este coordinador, utiliza relojes de lamport y el id del proceso, como un par ordenado de la forma $[LC_j, pid]$, se maneja una cola para garantizar la condición de entrega ordenada. En su primera fase P_i envía un mensaje usando multicast fiable, una vez enviado el resto de los procesos del grupo (no fallados) marca tentativamente los mensajes recibidos en base a su marca de lamport $[LC_j, id_j]$, finalmente para cada mensaje recibido por el proceso P_i , los asigna como **No comprometido**, (en la imagen, los mensajes con la N).

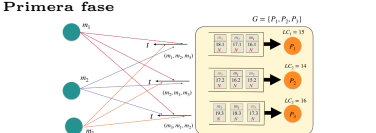


Figura 20: Primer paso ABCAST

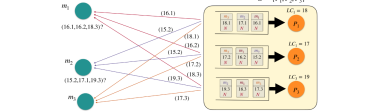


Figura 21: Segundo paso ABCAST

En la segunda fase, una vez P_i recibe la marca tentativa de sus miembros del grupo, marca la máxima marca tentativa como t_m , (en caso de que dos procesos tengan el mismo valor para el reloj, se desempata con el id). Finalmente, una vez definido el valor para el mensaje, se le notifica a todos los procesos del grupo de la marca ganadora y se pasa al valor de comprometido a **TRUE**.

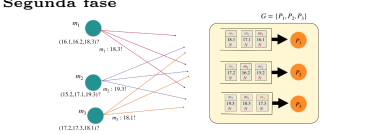


Figura 22: Tercer paso ABCAST

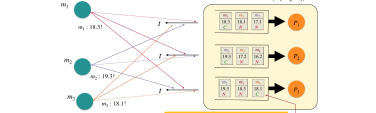


Figura 23: Cuarto paso ABCAST

Para establecer un número de mensajes por solicitud. Si hay N procesos, hay N envíos, como hay N procesos, hay N respuestas, y finalmente se tiene que comunicar para los N procesos N actualizaciones de marca para un mensaje, resultando finalmente en que la complejidad es $3N$.

CBCAST: Asegura orden causal, hace uso de relojes vectoriales, antes había un solo monitor, y cada miembro hace entrega causal. Las reglas son las siguientes. Para cada mensaje m_i , a ser enviado por P_i , se incrementa $VC_i[i]$. Cuando P_j recibe un mensaje m_i desde P_i , con la marca asociada al mensaje, entonces P_j retarda la entrega de m_i hasta que se cumpla la condición **FIFO** y la condición **CAUSAL** (estas se definieron en el capítulo 4, y están en este apunte).

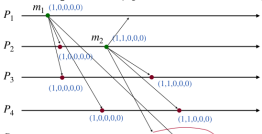


Figura 24: Ejemplo visual de CBCAST

Api de gestión de grupos: Para asegurar atomicidad nos provee una interfaz para gestionar grupos, servicios como: crear grupo, unirse a un grupo, salir de un grupo (por detección término o fallas), una lista de quienes reciben el mensaje (Vista) y notificar un cambio de en la lista del grupo.

Definimos un grupo atómico, un grupo donde de todos los miembros tienen la misma vista. Cuando existen cambios en la vista (alguien se sale o une al grupo), este proceso necesariamente debe ser atómico.

- Básicos:**
- $GID \leftarrow CreateGroup()$: crear un nuevo grupo
 - $JoinGroup(GID)$: unirse a un grupo
 - $LeaveGroup(GID)$: dejar un grupo o por detección de término o fallas
 - **Adicionales.** Además, para gestionar el protocolo:
 - $List \leftarrow Members(GID)$: Recibir lista de miembros (vista)
 - $SignalChanges(GID)$: señalar un cambio de vista

Obs.1 Notar que los 3 protocolos antes descritos tienen entrega atómica.

0.18 Algoritmo de PAXOS y otros (consenso con fallas de crash)

Paxos se usa para mantener consistente información del sistema. Como la información de la configuración del sistema. Permite coordinación tolerante a fallos. Paxos trabaja con un modelo de fallas de *crash* (no bizantinas), con omisión de mensajes. Trabajamos en una red con comunicación no fiable, y por paso de mensajes, los mensajes pueden sufrir retardos en la entrega (lo que hace que paxos sea parcialmente sincrónico), los mensajes pueden duplicarse, desordenarse o perder, pero no corromperse (modelo de falla de omisión). A nivel de tolerancia a fallas, solo soporta para $2f + 1 \leq n$.

- Roles de los actores:**
- **Proposer:** Propone un valor de un cliente para que sea aceptado por consenso.
 - **Acceptor:** Elige (o rechaza) alguna propuesta de un Proposer y les comunica su aceptación. Este componente tiene acceso a memoria estable (esto es debido a que debe recordar su decisión en caso de fallas)
 - **Learner:** Aprende cuál es el valor elegido por la mayoría de los Acceptors.
- Obs.1:** Un acceptor y un proposer puede estar en un mismo proceso, pero no pueden haber más de dos acceptor en un mismo proceso. Paxos cuenta con dos pasos esenciales:

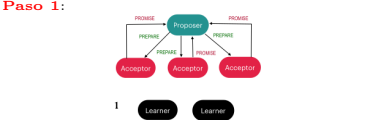


Figura 25: Primer paso de Paxos

Por el lado del **proposer**: Se propone una propuesta con un valor **VALUE**, y esta propuesta tiene un ID asociado (este id es estrictamente creciente).

Por el lado del **acceptor**: El acceptor verifica si el ID de la propuesta es mayor al que ya tiene (en caso de que no este no responde), para el caso donde si es el mayor, el ID recibido, será el nuevo ID máximo, si la propuesta no ha sido aceptada, este le responde al proposer con una promesa con el ID, caso contrario, este responde con una promesa con el ID, el ID aceptado, y el **VALUE** aceptado. **Paso 2:**



Figura 26: Segundo paso de Paxos

Fase 2A (Aceptación):

Por el lado del **proposer**: Si un proposer ha recibido promesas de la mayoría de los acceptors, comprueba si alguna respuesta incluye una propuesta previamente aceptada; si existe, adopta el **VALUE** de la propuesta con el ID más alto, en caso contrario mantiene su **VALUE** original. Finalmente envía a la mayoría de los acceptors un mensaje de aceptación con su ID y el **VALUE** seleccionado.

Por el lado del **acceptor**: En caso de que el acceptor reciba un mensaje de aceptación con un ID mayor o igual al máximo ID conocido, este informa a los Learners del valor aceptado.

Fase 2B (Aprendizaje):

Por el lado del **learner**: Cuando un learner recibe el mismo par (ID, **VALUE**) aceptado por una mayoría de acceptors, registra ese **VALUE** como el valor elegido (consenso) y lo entrega a la aplicación correspondiente.

Por el lado del **proposer**: Si un proposer recibe confirmaciones de aceptación (accepted) de una mayoría de acceptors para su propuesta (ID, **VALUE**), considera que el valor ha sido elegido y puede difundir el resultado a los learners o al cliente.

Proposer:

- **ID** : Identificador único y ordenado de una propuesta.
- **cnt** : Mayor ID conocido en el sistema (podría ser un TS o contador con UID)

- Acceptor:**
- **max_id** : Máximo identificador de propuesta conocido por Acceptor y que respondió con una promesa (en 1ª fase).
 - **proposals_accepted** : El Acceptor ha aceptado una propuesta (en 2ª fase), una variable booleana.
 - **accepted_ID** : ID de la propuesta aceptada (en 2ª fase).
 - **accepted_VALUE** : Valor de la propuesta aceptada (en 2ª fase).

El quorum para aceptar un valor es de $\lfloor \frac{n}{2} \rfloor + 1$

Capítulo 7: Replicación de Datos

0.19 Introducción a la Replicación de Datos

Escalabilidad/desempeño: La escalabilidad de sistemas de almacenamiento de datos con buen desempeño ante cargas se basa en dos técnicas:

- **Particionamiento (sharding o fragmentación):** Datos se fragmentan y se distribuyen localizandolos en varios sitios, aumentando el desempeño al distribuir la carga y agregar recursos
- **Replicación:** Datos replicados mejora el desempeño al distribuir cargas entre réplicas, pero también al acceder a una copia más cercana (mejor tiempo de respuesta y menor latencia)

Disponibilidad/Resiliencia: Datos replicados para alta disponibilidad y tolerancia a fallos

Por ejemplo, en un modelo de fallas fail-stop, si α_0 es la disponibilidad de una réplica, entonces la disponibilidad de n réplicas es: $\alpha_n = 1 - (1 - \alpha_0)^n$

Cuando existen actualizaciones el **gran desafío es mantener la consistencia** entre las réplicas

- Desafíos de la replicación de datos:**
- **Concurrencia:** Actualizar una copia genera una inconsistencia entre ellas. Se requiere coordinar acceso concurrente para garantizar consistencia.
 - **Fallas:** En ambientes distribuidos se producen fallas de servidores y/o comunicación. Se requiere coordinar recuperación ante posibles errores para asegurar estado consistente y mantener disponibilidad del sistema.
 - **Particiones de red:** Si se produce una partición de la red, no es posible coordinar a los nodos, pertenecientes a diferentes particiones. Se requiere coordinar la consistencia entre réplicas durante una partición, lo que requiere sacrificar disponibilidad.
 - **Complejidad:** La ejecución de los diferentes

protocolos para consistencia y disponibilidad puede ser demasiado costoso, sacrificando desempeño y limitando la escalabilidad. Es decir, mayor latencia, uso de ancho de banda y carga sobre servidores

- 0.19.1 Modelos de replicación**
- Semántica de consistencia:**
- **Consistencia fuerte:** Asegura que todas las réplicas retornan el mismo valor.
 - **Consistencia débil:** Permite retornar diferentes valores, sacrificando consistencia para favorecer alta disponibilidad y mejorar desempeño. (consistencia eventual)
- Atomicidad computacional:**
- **Mono-objeto + mono-operacional:** Cada operación sobre un objeto de datos se realiza atómicamente, independiente de otras operaciones y objetos. (en ejecución de métodos en objetos distribuidos y replicados)
 - **Multi-objeto + Multi-operacional (transaccional):** Operaciones de diferentes transacciones actualizan concurrentemente las réplicas. (con propiedades ACID)

0.19.2 Consistencia débil

La consistencia débil es útil para mejorar disponibilidad y desempeño, a costa de sacrificar consistencia

- Ejemplos:**
- **DNS:** Actualizaciones desde un único primario son propagadas asíncronamente a los secundarios, no siendo inmediatamente visibles.
 - **WWW:** Uso de caching intensivo, porque es tolerable tener una página desactualizada. Peder haber copia de caché desactualizadas en navegadores, servidores proxies o CDN.
 - **Documentos compartidos:** Actualización de un documento (replicado) quedará después de un tiempo consistente y se resolverán conflictos por actualizaciones concurrentes. Similar situación en CVS o GitHub.
 - **Carrito de compra:** Copia local puede quedar desactualizada por desconexión.

0.19.3 Teorema CAP

Teorema: Un sistema de datos compartidos puede cumplir simultáneamente a lo más dos de las siguientes tres propiedades:

- **C - Consistencia de datos:** Equivalente a mantener actualización de una copia.
- **A - Disponibilidad (Availability):** Tolerancia a fallos y del sistema para seguir actualizando las copias de datos.
- **P - Tolerancia a particiones:** Sistema sigue operando, pese a existir una partición de red.

C	A	P	ANÁLISIS
✓	✓	✗	Si no se tolera una partición de red, entonces existe siempre comunicación suficiente para mantener la consistencia y la disponibilidad. No se puede tener un sistema que mantenga las tres propiedades (p.e., un modelo de fallas de crash de procesos y canales de mensajes).
✓	✗	✓	Si se tolera una partición de red y se quiere alta disponibilidad, entonces se debe sacrificar la consistencia. No se puede tener un sistema que mantenga las tres propiedades (p.e., un modelo de fallas de crash de procesos y canales de mensajes).
✗	✓	✓	Si se tolera una partición de red y se quiere alta disponibilidad, entonces se debe sacrificar la consistencia. No se puede tener un sistema que mantenga las tres propiedades (p.e., un modelo de fallas de crash de procesos y canales de mensajes).

Figura 27: Teorema CAP

0.19.4 Estrategias de actualización de réplicas

- Sincronización de actualizaciones:**
- **Sincrónica:** Una actualización está completa cuando se obtiene confirmación de todas para comprometerla. Asegura una consistencia fuerte pero introduce mayor latencia.
 - **Asincrónica:** Una actualización se considera completa si la confirma un nodo distinguido. Demás réplicas se actualizan más tarde, lo que reduce latencia y mejora desempeño. Mejora disponibilidad, pero puede producir inconsistencias por concurrencia, latencias o fallas.
- Propagación de actualizaciones:**
- **Anísota:** Las actualizaciones se propagan inmediatamente una vez comprometidas. Se mantiene réplicas cercanamente sincronizadas.
 - **Perezosa:** Actualizaciones se envían después de cierto retardo o tiempo. Habilita replicación optimista y operaciones desconectadas.

Aspecto	Esquema centralizado	Esquema descentralizado
Control	Un nodo (el coordinador) gestiona actualizaciones y controla el flujo de datos.	Todos los nodos comparten responsabilidades, sin punto central de control.
Coordinación	Coordinador central gestiona y valida actualizaciones.	No se necesita un coordinador central. Se usa para otras operaciones que no sean de actualización.
Consistencia	Se tiene un único punto de control (el coordinador) para validar las actualizaciones.	Se tiene un consenso distribuido (p.e., consenso por mayoría o consenso por quórum).
Propagación de actualizaciones	Se tiene un único punto de control (el coordinador) para propagar las actualizaciones.	Se tiene un consenso distribuido (p.e., consenso por mayoría o consenso por quórum).
Fallas	Existe un punto único de falla (p.e., el coordinador). Si falla, el sistema se detiene.	No existe un punto único de falla. Si falla un nodo, el sistema sigue funcionando.
Resiliencia	El sistema es vulnerable a fallos de coordinación y de comunicación.	El sistema es más resiliente a fallos de coordinación y de comunicación.
Ejemplos	Base de datos centralizada (p.e., Oracle, MySQL).	Base de datos distribuida (p.e., MongoDB, Cassandra).

Figura 28: Descentralización en la Gestión de réplica

0.19.5 Estrategias de Resolución de conflictos

- Pesimista:** Estrategia de replicación que previene conflictos, coordinando actualizaciones antes de comprometerlas.
- Garantiza una **consistencia fuerte** en replicación de datos.
 - Se sacrifica disponibilidad, si se tolera particiones de red. Más pesado y no escala bien.
- Técnicas:** Uso de métodos de candados, protocolos de quorum o consenso
- Optimista:** Permite actualizaciones concurrentes, baja la premisa que los conflictos rara vez ocurren; y si ocurren, se resuelven más tarde.
- Acepta semánticas de **consistencia débil**. Típicamente las actualizaciones se propagan asíncronamente. Requiere mecanismos de resolución de conflictos.
 - Se logra **mayor disponibilidad**, sacrificando **consistencia**, eventualmente tolerando particiones de red. Escala bien, con mejores tiempos de respuesta.
- Técnicas:** Uso de métodos como relojes vectoriales, marcas de tiempos, algoritmos epidémicos.

0.20 Modelos semánticos de consistencia en replicación

Definición (Modelo de consistencia): Contrato entre los procesos y los almacenes de datos respecto a cómo se realizan las operaciones sobre los objetos de datos replicados. **Involucra** reglas sobre cómo y cuándo las actualizaciones sobre datos replicados se vuelven visibles y qué garantía de orden existe para las operaciones observables por los clientes.

0.20.1 Consistencia estricta
Cualquier lectura sobre un objeto de dato x retorna el valor correspondiente al resultado de la escritura más reciente sobre x .
Nomenclatura: $W_i(x)$: a escritura del proceso P_i del valor a en el dato x . $R_j(x)$: a lectura del proceso P_j del valor a desde el dato x .

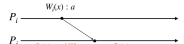


Figura 29: Consistencia estricta

0.20.2 Consistencia fuerte (1-copia)

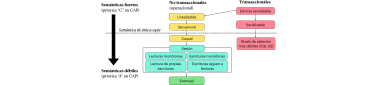
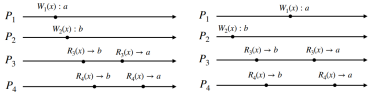


Figura 30: Modelo de consistencia en replicación

- Consistencia linealizable: Una vez se completa una escritura sobre un objeto x , toda lectura posterior sobre x deben retornar el valor de esa escritura, o el de una escritura posterior. Una vez que una lectura sobre x retorna un valor particular, todas las lecturas posteriores sobre x retornan ese valor, o el de una escritura posterior.
- Consistencia serializable: Garantiza que la ejecución de un conjunto de transacciones con operaciones sobre varios objetos de datos, es equivalente a alguna ejecución serial. Corresponde a semántica de serialización de una copia.
- Consistencia estrictamente serializable: Es una consistencia linealizable y serializable.

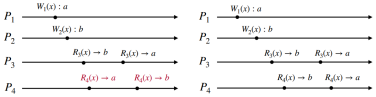
0.20.3 Modelos de consistencia no-transaccional

A) Consistencia linealizable: El resultado de cualquier ejecución es la misma, si las operaciones sobre los datos fueron ejecutadas en algún orden secuencial por todos los procesos y las operaciones de cada proceso aparecen en esa secuencia en el orden especificado por el programa. El orden es consistente con el tiempo real en la que ocurrió cada operación durante la ejecución. Requiere sincronización de todas las copias para cada actualización.



(a) Sin consistencia linealizable (b) Con consistencia linealizable

B) Consistencia secuencial (No-transaccional): El resultado de cualquier ejecución es la misma, si las operaciones sobre los datos fueron ejecutadas en algún orden secuencial y las operaciones de cada proceso individual aparecen en esa secuencia en el orden especificado por el programa. Supone que operaciones de diferentes procesos se entremezclan y todos los procesos ven este mismo orden. No hay restricciones temporales, se cumple especificación de una única copia del objeto de datos. Corresponde al modelo de múltiples procesadores con memoria compartida.



(a) Sin consistencia secuencial (b) Con consistencia secuencial

C) Consistencia causal: Escrituras que son potencialmente causales tienen que verse en todos los procesos en el mismo orden. Escrituras concurrentes pueden verse en diferente orden en distintos procesos. No se cumple especificación de una única copia para un objeto de datos. Se puede implementar usando relojes vectoriales. Supone una consistencia más débil que consistencia secuencial, pues sólo impone un ordenamiento para eventos causales.

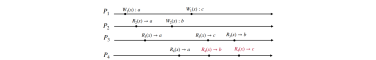


Figura 33: Causal pero no secuencial

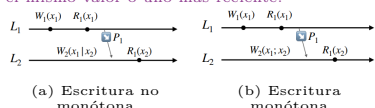
- Problemas de consistencia fuerte:**
- Disponibilidad y latencia de red: Internet es muchas veces lenta y poco fiable, afectando la disponibilidad y latencia, los dispositivos móviles se desconectan frecuentemente de la red. Intento de sincronización con un servidor no disponible puede provocar bloqueos.
 - Escalabilidad de sistemas replicados: En redes extendidas con gran cantidad de nodos, es difícil replicar masivamente sistemas que realizan frecuentemente actualizaciones. Throughput y disponibilidad empeora con el incremento del número de nodos.
 - Necesidades de aplicaciones de trabajo colaborativo: Se comparten datos, pero permitiendo trabajar temporalmente en forma aislada. Es preferible actualizar datos independientes y resolver ocasionalmente conflictos cuando ocurren.
- D) Consistencia eventual:** En ausencia de nuevas operaciones de escritura sobre un objeto de dato x , todas las operaciones de sobre x eventualmente retornarán el mismo resultado. Requiere que las actualizaciones se propaguen a todas las réplicas, tal que en algún momento todas convergen al mismo valor. Es más escalable y barato de implementar. Funciona bien mientras los clientes accedan a una misma copia.

0.20.4 Modelos de consistencia de sesión
Concepto de consistencia: Solo provee garantía de consistencia para un único cliente. No provee garantía de consistencia si existe acceso concurrente de diferentes clientes a un mismo dato. Potenciales conflictos se deben resolver en una capa superior.
Autoría: Base de datos para clientes móviles, un cliente trabaja con una copia local, cada dato tiene un dueño quien preferentemente lo modifica.

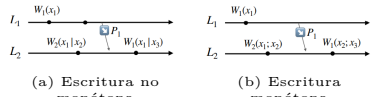
les, un cliente trabaja con una copia local, cada dato tiene un dueño quien preferentemente lo modifica.

Notación: $R_1(x_2)$: P_1 lee versión x_2 del dato. $W_1(x_2)$: P_1 escribe versión x_2 del dato sin conocer otras versiones anteriores. $W_1(x_1; x_2)$: P_1 escribe versión x_2 del dato, basada en versión x_1 . $W_1(x_1; x_2)$: P_1 escribe versión x_2 del dato concurrentemente a x_1

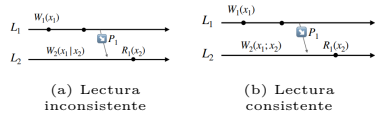
A) Lecturas monótonas: Si un proceso P lee un valor de un objeto de datos x , cualquier lectura posterior de P sobre x siempre retornará el mismo valor o uno más reciente.



B) Escrituras monótonas: Operación de escritura de un proceso P sobre un objeto de datos x se completa antes que cualquier operación posterior de escritura de P sobre x . Las operaciones de escritura, independiente del sitio, se propagan en el orden correcto a todas las copias de un almacén de datos.



C) Lecturas de propias escrituras (read your writes): Los efectos de una operación de escritura de un proceso P sobre un objeto de datos x va a ser siempre visto por operaciones de lectura sucesivas sobre x por P .



D) Escrituras siguen lecturas (writes follow reads): Una operación de escritura de un proceso P sobre un objeto de datos x que sigue a una operación de lectura sobre x realizada también por P , tiene garantizado que P va a obtener el mismo valor de x o uno más reciente.



0.21 Arquitectura para replicación de datos



Figura 38: Modelo básico de referencia
Ejemplos de sistemas con replicación de datos: Sistemas transaccionales, de consistencia eventual, máquina de estado replicada, sistemas de multicasting, sistemas móviles con caching, sistemas de blockchain.
FE es un proxy del cliente al servicio, cohabita con el cliente en un único proceso. Mediante las APIs hace transparente a los clientes el acceso y la replicación de datos.
RM es servidor que gestiona una réplica y coordina con otros RMs para consistencia. Grupo de RMs puede ser estático o dinámico. Un RM puede sufrir fallas de crash y/o omisión. Se asume algún grado de sincronismo.

0.21.1 Replicación pasiva con protocolo Primario-Respaldo (Primary-backup)



Figura 39: Protocolo de Primario-Respaldo

- Petición:** FE genera una petición con un identificador único (UID) y lo envía al Primario. Peticiones se generan sincrónicamente por el cliente (secuencial).
 - Coordinación:** El primario procesa cada petición atómicamente y en el orden de llegada. Antes verifica el UID de la petición para asegurarse que no ha sido procesada previamente (at-most-once). Si es el caso, responde inmediatamente con copia de respuesta guardada en buffer.
 - Ejecución:** El primario ejecuta la petición y almacena en buffer la respuesta.
 - Acuerdo:** Si la petición es una actualización, el primario envía a todos los RMs de respaldo el estado de la actualización, la respuesta y UID. Cada respaldo responde con un ACK al primario.
 - Respuesta:** El primario responde a FE, que a su vez le responde de vuelta al cliente. Las operaciones de lectura se hacen directamente sobre cualquier réplica para distribuir carga y aumentar disponibilidad. La propagación de actualizaciones es asíncrona y perezosa, para mejorar el tiempo de respuesta (sacrificando consistencia).
- El sistema implementa una semántica de consistencia linealizable. Si el primario falla, un respaldo debe asumir su rol. Es bastante pesado, para mejorarlo se requiere sacrificar consistencia. Para balancear la carga, los clientes pueden leer directamente en algún servidor de respaldo, pero se pierde consistencia linealizable y se conserva consistencia secuencial. En un modelo Fail-Stop, se requiere de $f+1$ copias; en modelo crash se requiere $2f+1$ copias; en modelo bizantino se requiere $3f+1$ copias.
- Reparación:** Tres enfoques:
- Hot backup:** Respaldo siempre listo para tomar control si falla el primario, con retardo mínimo.
 - Warm backup:** Se ha realizado el trabajo

de preparación para tomar el control, pero no está listo inmediatamente para asumir el rol de primario.

- Cold backup:** No existe trabajo preparatorio, se debe recuperar la base de datos antes de que el respaldo pueda tomar el control.
- Detección de fallas:** Para detectar la falla del primario, debe haber un proceso de monitoreo que deba ser fiable en la detección. Puede haber retardo en detectar la falla, lo que aumenta el MTRP y degrada la disponibilidad. Si falla el primario, es posible que al haber fallado y reparado quiera tomar el rol de primario haciendo fallar al primario suplente.



Figura 40: Esquema de Primario-Respaldo

0.21.2 Replicación activa



Figura 41: Esquema de Replicación Activa

- Petición:** FE genera una petición con un identificador único y lo envía a todos los RMs. FE a lo más falla por crash.
 - Coordinación:** El sistema de comunicación grupal entrega la petición a cada RM en el mismo orden.
 - Ejecución:** Cada RM ejecuta la petición y en el mismo orden. Grupo de RMs define una máquina de estado replicada y determinista. La respuesta contiene UID de la petición.
 - Acuerdo:** No se requiere. Está subyacente en el protocolo de multicast.
 - Respuesta:** Cada RM envía su respuesta al FE, el cual debe filtrar respuestas repetidas según el mismo UID o modelo de fallas usado.
- Se logra consistencia secuencial. Peticiones de un mismo FE se procesan en orden FIFO. No es linealizable. Si los clientes no se comunican entre sí mientras esperan una respuesta se garantiza ordenamiento causal. Si tiene multihebra en los clientes, puede ocurrir comunicación entre ellos y si se quiere orden causal habrá que agregar un protocolo de multicast que provea el orden causal y total.
- Puede tolerar f fallas bizantinas, si se tiene al menos $2f+1$ réplicas y FE debe recolectar al menos $f+1$ respuestas iguales. Es posible mejorar el desempeño si se considera que operaciones de diferentes clientes conmutan.

0.22 Protocolos de consistencia de datos
Tipo de protocolos: Positivista (prevención de conflictos) y optimista (actualizaciones concurrentes y con resolución de conflictos).

0.22.1 A) Protocolo de copia primaria

- Actualización:** Reservar la copia primaria, modificar la copia primaria. Si modificación es válida, copia primaria provocará actualización de copias secundarias.
 - Consultas:** Consulta consistente: Reservar en copia primaria y luego consultar copia local. Consulta aproximada: Consultar la copia local, sin reservar la primaria.
- Si existen $k+1$ copias, se pueden tolerar k fallas de nodos. Si el nodo primario falla, se puede activar algoritmo de elección, si se produce partición, sólo se permiten operaciones en la partición del primario. Funciona bien si es posible discriminar entre falla del primario y partición de red. Existen esquemas que detectan si el primario está en una partición con una mayoría de nodos; si no es el caso, se le destituye y una partición mayoritaria puede elegir otro primario.

0.22.2 B) Protocolo de Quorum mayoritaria

- Antecedentes:** Permite tolerar fallas de nodos y particiones de red, sin necesidad de discriminar entre ellas.
 - Idea básica:** Asignarle a cada copia o réplica de dato una cierta cantidad de votos. Una operación de lectura o escritura sólo se acepta si el usuario logra obtener suficientes votos.
- Quórum de observación:** $V(a)$: Número de votos totales sobre el dato «a». $R(a)$: Quórum necesario para leer «a». $W(a)$: Quórum necesario para poder escribir «a».
- Relaciones a cumplir:** $2 \cdot W(a) > V(a)$: evita actualizaciones simultáneas. $W(a) + R(a) > V(a)$: evita lectura y escritura simultánea.

0.22.3 C) CODA: Sistema de archivos distribuidos

- Existen varios servidores que mantienen réplicas de archivos. Clientes pueden trabajar desconectado, manteniendo en caché copia de archivos de trabajo. Replicación optimista para detectar conflictos. En la reconexión se detectan conflictos y se resuelven semi-manualmente. Sus conceptos son patrones de diseño claves en sistemas móviles y colaborativos.
- Mecanismo de resolución de conflictos:** Existen n sitios, donde existe una copia de cada archivo. Se mantiene un vector n -dimensional V_i para cada archivo en cada sitio i ; cada sitio i cuenta el número de actualizaciones provenientes de cada otro sitio j en $V_i[j]$. Si no existe partición, entonces los vectores son iguales en todos los sitios; sino pueden diferenciarse en diferentes particiones. Si existe casualidad, se dice que un vector domina al otro, sino existe conflicto.
- V_i domina a V_j :** Se actualiza copia de sitio i a sitio j .
 - V_i en conflicto con V_j :** El administrador del sistema realiza resolución (semi)-manual.

0.22.4 CODA

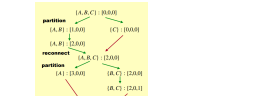


Figura 42: CODA

0.22.5 D) Protocolos epidémicos (optimista y sin líder)

- Implementan consistencia eventual, propagando actualizaciones asíncrona, entre los RMs. No resuelven conflictos de actualización, que deber ser realizado por otra instancia de orden superior. Actualizaciones se propagan infectando progresivamente las réplicas hasta infectarlas todas.
- Tres tipos (o estados) para las réplicas en los servidores (RM):
- Infectado:** Tiene actualización que desea.
 - Propenso:** No ha sido infectado.
 - Removido:** Está actualizado, pero no propaga más la actualización.
- Idea del algoritmo:** Actualizaciones se realizan inicialmente en una o pocas réplicas. Una réplica propaga sus actualizaciones a un número limitado de vecinos. Las actualizaciones se propagan flojamente, en forma asíncrona y no se realizan inmediatamente, lo que se puede demorar más en lograr consistencia eventual. Finalmente, cada actualización alcanzará a cada réplica, quedando todas ellas con un mismo valor.
- Estrategias de actualización:**
- Anti-entropía:** Cada servidor de réplica regularmente elige a otra aleatoriamente Q , e intercambia diferencias de estados, dejándolos iguales al posteriormente. Dos estrategias de intercambio. 1. Push: P sólo transfiere sus actualizaciones a Q (sin conocer las de Q). 2. Pull: P sólo obtiene nuevas actualizaciones desde Q (sin comunicar las propias). 3. Push & Pull: P y Q intercambian todas sus actualizaciones.
 - Chisme:** Réplica que acaba de actualizarse, lo comunica a otras. Si un servidor P es actualizado, contacta aleatoriamente a un servidor Q arbitrario y tratar de empujar su actualización. Si Q ya está actualizado, P pierde interés en propagar la actualización. Para asegurar que se actualizan todas las réplicas se combina con Anti-Entropía.

Caso blockchain:

- Arquitectura:** Arquitectura 2P2, descentralizada, simétrica y sin líder. Cada nodo mantiene copia de la cadena de bloques.
- Propagación de actualización:** Cuando un nodo resuelve el problema de minado, lo propaga al resto de la red. Minero ganador asume el rol de coordinador para propagar nuevo bloque a ser agregado a la cadena. Se usa un método optimista con un estilo epidémico de chisme para propagar (consistencia eventual).
- Resolución de conflictos:** Se produce un fork, si se agregan concurrentemente varios bloques. Se resuelve por la regla de la cadena más larga es válida.

0.23 Ubicación y distribución de réplicas

0.23.1 Gestión de número de réplicas
Estrategias de ubicación y colocación de réplicas: Distribución geográfica y aspectos de balance de carga, latencia, consumo energético. **Dinamismo en número de réplicas:** Replicación elástica por demanda, recolección de basura, variación de miembros.

0.23.2 Localización de réplicas



Figura 43: Localización de réplicas

0.23.3 Réplicas lado del servidor

- Réplicas permanentes:**
- Característica:** Corresponde a conjunto inicial de réplicas, normalmente un número pequeño y estático de réplicas.
 - Dos estrategias:** Clustering (balance de carga) y Mirroring (mayor distribución geográfica).
 - Ejemplos:** Sitio web replicado en cluster, base de datos distribuidas o replicadas.
- Réplicas iniciadas por el servidor:**
- Característica:** Copia réplica a lugar más cerca del cliente, para satisfacer la alta demanda, de preferencia de lectura.
 - Gestión de N° de réplicas:** Requiere monitorear frecuencia de acceso. Se copia si se sobrepasa un umbral, se borra si baja de un umbral y las copias también pueden migrar.
 - Ejemplo:** El CDN

0.23.4 Réplicas al lado del cliente

Conocidas como (client) caches. Frecuentemente administradas por el cliente, pero a veces se incorpora al servidor, mejora tiempo de acceso y típicamente se utiliza una marca de tiempo de último acceso.

En caso de caching: Se propaga notificaciones de actualización, datos de una copia a otra, operación de actualización a otras copias.



Figura 44: Propagación de actualizaciones

Capítulo 8: Transacciones Distribuidas
0.24 Concepto de Transacción
Definición: Una transacción es un conjunto de acciones que realiza múltiples operaciones sobre varios objetos de datos y que produce cambios de estado consistente en el sistema. Entre cambios concurrentes ocurren aun en presencia de accesos concurrentes y con las posible ocurrencia de errores o fallos. Los cambios de estado ocurren cuando una transacción se compro-

mete; pero si no ocurre, aborta.

0.24.1 Aplicaciones de las transacciones

- Bases de datos.
- Base de datos distribuidas.
- Sistemas de archivos distribuidos.
- Aplicaciones tolerante a fallos: Proveen acciones y cambios atómicos para enmascarar fallos.

0.24.2 Transparencia de Transacciones

Provee dos tipos de transacciones: **Concurrencia y fallos**. Pero si es distribuido, también provee: Transparencia de **ubicación y replicación** (si existe).

Se usa buffering para proveer alto desempeño en acceso de datos persistentes, lo que puede considerarse como transparencia de **persistencia**.

Propósito: Permite simplificar el modelo de programación de aplicaciones en ambientes complejos de operación que trabajan con datos persistentes.

Problemas básicos: **Control de concurrencia:** ¿Cómo hacer transparente al usuario el hecho de que los datos pueden ser accedidos concurrentemente? (transparencia de concurrencia). **Recuperación de errores:** ¿Cómo ocultar al usuario la concurrencia de errores? (Transparencia de fallos o tolerancia a fallos).

0.24.3 Modelo de Fallos (típicos)

- **Modelo de almacenamiento de datos:** Operaciones de escritura sobre el medio de almacenamiento pueden fallar. Lecturas pueden usar un código tipo checksum para detectar errores de lectura. El medio puede decaer perdiéndose información, o puede destruirse completamente.
- **Procesos:** Pueden caerse ocasionalmente, perdiendo la memoria volátil. Procesos fallados se detienen (fail-stop, crash), sin mandar mensajes o escribir en disco datos erróneos. Existe procedimiento de recuperación basado en el uso de memoria estable, para restaurar estado anterior a la falla y recomenzar recuperación desde un estado consistente.
- **Comunicación:** Mensajes pueden sufrir retardo arbitrario, perderse, duplicarse, o corromperse. Receptor es capaz de detectar estos errores y recuperarse.
- **Memoria Estable:** Soporta operaciones atómicas de escritura sobre un medio persistente independiente de alta fiabilidad, capaz de sobrevivir caídas de proceso y fallos de medio de almacenamiento, para apoyar procedimientos de recuperación de errores en diversos escenarios.

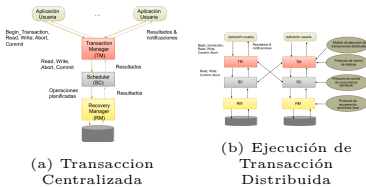
0.24.4 Modelo transaccional plano

Ejecución atómica y fiable en presencia de fallos, ejecución correcta en presencia de múltiples accesos de usuarios y gestión correcta de réplicas.

0.24.5 Propiedades ACID

- **Atomicidad:** La transacción se ejecuta completamente o no se ejecuta.
 - **Consistencia:** La base de datos pasa solo de un estado consistente a otro estado consistente o mantenerse en estado actual si se aborta.
 - **Aislación:** Una transacción no debe ser afectada por otras transacciones concurrentes.
 - **Durabilidad:** Los resultados de una transacción deben ser permanentes, sobreviviendo a fallos del sistema.
- Expandiendo el concepto de **Aislación**:
- **Serialización:** Si varias transacciones son ejecutadas concurrentemente, el resultado debe ser el mismo si se hubiera ejecutado serialmente en algún orden. **No se aceptan interferencias entre transacciones.**
 - **Resultados incompletos:** Una transacción incompleta no puede revelar sus resultados a otras transacciones antes de su compromiso.

Necesario para evitar abortos en cascada.



0.25 Control de Concurrencia de Transacciones



Figura 46: Modelo arquitectónico de un sistema de gestión de transacciones distribuidas

0.25.1 Planificación

Secuencias de operaciones ejecutadas por una o más transacciones.

0.25.2 Ejecuciones Seriales y Concurrentes

Definición: Ejecución serial: Un par de transacciones T_i y T_j se ejecutan serialmente en una planificación S ssi:

- Última operación de T_i precede a primera operación de $T_j \nexists$ última operación de T_j precede a primera operación de T_i .
- Si $\forall T_i, T_j \in S : T_i$ y T_j se ejecutan serialmente. Entonces la planificación S es serial y se cumple el predicado $Serial(S)$.

Ejecución concurrente: Si un par de transacciones no son ejecutadas serialmente, entonces se dice que su ejecución es concurrente. Si en una planificación S existe un par de transacciones que se ejecutan concurrentemente, entonces la ejecución de S es concurrente. Es decir, $Serial(S)$ es falso.

0.25.3 Serialización de transacciones

Axioma: Una ejecución serial de transacciones es siempre correcta

Principio de serialización: Un método de control de concurrencia es correcto si es serializable. Es decir, existe una secuencia equivalente en que las operaciones de cada transacción aparecen antes o después de otra transacción, pero no entremezcladas.

0.25.4 Equivalencia de planificaciones concurrentes

Condiciones de equivalencia:

- Cada transacción lee para cada dato el mismo valor y realiza, por lo tanto, la misma computación.
- La última operación de escritura sobre cada dato es realizada por la misma transacción.



Figura 47: Lectura y escritura de datos

0.25.5 Conflicto entre operaciones

Operaciones en conflicto: Dos operaciones están en conflicto, ssi:

- Operan sobre los mismos datos
- Una de las operaciones es de escritura
- Cada operación pertenece a diferentes transacciones

Teorema de equivalencia: Dos planificaciones S_1 y S_2 son equivalentes, ssi: $\forall o_{a_i}, o_{a_j}$ en conflicto: $(S_1 : o_{a_i} < o_{a_j}) \Rightarrow (S_2 : o_{a_i} < o_{a_j})$

Dos planificaciones son equivalentes si respetan

el orden de precedencia para todas las operaciones en conflicto. Operaciones que no están en conflicto pueden permutarse, sin afectar el resultado de equivalencia. Es decir, una planificación serializable podría ser equivalente a diferentes planificaciones seriales.

0.25.6 Protocolo de candado de dos fases (2PLP)

Reglas:

- Una transacción antes de acceder a un objeto debe ponerle **candado**.
- Una transacción **no puede adquirir otro candado si ha liberado alguno**.

Fases: La segunda regla determina que el protocolo tiene dos fases.

- **Fase de crecimiento: acumula candados**
- **Fase de decrecimiento: libera candados**

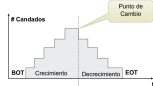


Figura 48: 2PLP - simple

Se puede demostrar que 2PLP serializa (al menos) el orden en que se producen los cambios de fases. El principio de la demostración es que aquella transacción no tiene conflicto con ninguna otra transacción.

0.25.7 Tipos de candado

Para aumentar el grado de concurrencia, se usan diferentes tipos de candado.

Esquema candado exclusivo y compartido:

- **Candado exclusivo (De escritura)**
 - **Candado compartido (De lectura)**
- Un dato puede tener múltiples candados de lectura de diferentes transacciones, **pero sólo uno de escritura**.

0.25.8 Problemas con 2PLP

Abrazo mortal (deadlock): Soluciones: Timeout con aborto y reinicio o detección de deadlock.

Aborto en cascada: Soluciones: Usar 2PLP estricto, restringe más la concurrencia, pero evita los abortos en cascada.



Figura 49: 2PLP - estricto

0.26 Control de concurrencia en transacciones distribuidas

0.27 Modelo de transacciones distribuidas

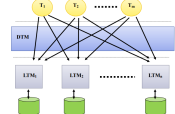


Figura 50: Modelo de transacciones distribuidas

Nomenclatura: T_i : Transacción $N^{\circ}i$, LTM: Local Transaction Manager, DTM: Distributed Transaction Manager.

0.27.1 A) 2PLP distribuido

Procedimiento: Los candados son administrados localmente en cada sitio usando 2PLP. A nivel de DTM, cada solicitud se hace a través del coordinador, quien la redirige algún agente si fuese necesario.

Correctitud: En cada sitio la planificación es serializable. Debe existir un ordenamiento global, de manera que las serializaciones locales respeten ese orden global.

0.27.2 B) Protocolo de ordenamiento por marcas de tiempo básico (BTO)

Cada transacción al iniciarse recibe un TS como identificador en su lugar de origen. Cada operación (R o W) lleva el TS de la transacción. **Para cada dato x se ejecuta el mayor TS de lectura y de escritura**, que denotaremos respectivamente por: $RTM(x)$ y $WTM(x)$

Reglas de BTO

- **Regla de lectura:** Sea TS_i la marca de una operación de lectura sobre x . Si $TS_i < WTM(x)$, se rechaza la lectura y se reinicia la transacción. Si no, se realiza la lectura y se actualiza $RTM(x) = \max(RTM(x), TS_i)$
- **Regla de escritura:** Sea TS_j la marca de una operación de escritura sobre y . Si $TS_j < RTM(y)$ o $TS_j < WTM(y)$, se rechaza la escritura y reinicia transacción. Si no, se realiza la escritura diferida y $WTM(y) = TS_j$

El protocolo **serializa en el orden creciente de las marcas de tiempo**, operaciones originadas por marca de tiempo creciente. El protocolo **no produce deadlock**, al no existir bloqueos y ciclos de espera las transacciones que recomienzan no dejan huella y parten con un nuevo TS (que debe ser mayor). Puede producir postergación indefinida. **Tiene un costo significativo en memoria** para mantener información sobre TS asociado a última lectura y última escritura de cada objeto de datos.

0.27.3 C) Métodos optimistas

Principio de funcionamiento: Asumen que los conflictos durante la ejecución de una transacción son poco probables, de manera de reducir el costo de estar sincronizando cada acceso a datos. Se valida al término de una transacción si existieron conflictos durante su ejecución, si no existe conflictos, se compromete; si no, se aborta. **Serializa al menos en el orden que se van validando las transacciones.**

Fases de un método optimista:

- **Fase de lectura:** Cuerpo de la transacción, copiando datos usados desde la memoria persistente (BD) a buffers en la memoria principal y volátil. Copias que puede ser actualizada en memoria principal, pero sin respaldar a la BD
- **Fase de validación:** Se comprueba que no existan conflictos.
- **Fase de escritura:** Si no hay conflicto se compromete la transacción, si no es así, se aborta aplicando rollback.

0.27.4 Algoritmo de Kung

Suponiendo que T es una transacción donde T_s es mayor número de secuencia al comenzar T y T_f el mayor número de secuencia al comienzo de cada fase de validación.

Algoritmo:

- **Asume validez**
- **Busca conflictos**
- **Decisión:** Si hubo cambios declara la transacción como inválida. Si no hubo cambios, se guardan los cambios.

0.27.5 Casos de control de concurrencia

- **Dropbox:** Usa un **método optimista** con un historial de versiones. Conflictos son resueltos manualmente por una transacción rechazada.
- **Dynamo:** Servicio de Amazon, utiliza un **método optimista** de control de concurrencia y replicación. Se deben reconciliar versiones en conflicto.
- **Google:** Permite acceder a documentos concurrentemente. Procesador de palabras ve el cursor y actualizaciones de otro usuario.